

1.5 Multi-Access Edge Computing (MEC)

Multi-access Edge Computing, as defined by European Telecommunications Standards Institute (ETSI), is a method for deploying compute resources close to the device. This provides for lower round trip times between the device and the server performing the processing, and moves the bulk of compute out of the core network. During this project, the Multi-access Edge Computing server is going to run a Python Flask application with Socket.IO and implement a UDP listener that listens for log data sent from a Raspberry Pi over the 5G cellular network, and deliver it in real-time to a web browser.

1.6 Motivation

- Increasing need for industrial systems to have real-time telemetry available, while standard Linux cannot ensure this.
- Even though 5G has the potential to deliver low latencies, the delay from the application layer to the physical layer may be impacted much more by the OS scheduling process (jitter), than the network path itself.
- No published literature on how to combine PREEMPT_RT with a 5G SA with native MEC network using a Raspberry Pi
- This project will create a complete practical (end-to-end) reference implementation of real-time edge telemetry.

1.7 Problem Statement

Low-latency and **deterministic communications** are both requirements of modern industrial and embedded products to perform successfully. 5G Standalone networks are able to provide high throughput communications and advanced capabilities such as network slicing and Multi-access Edge Computing, but do not guarantee stable real-time performance. This is due to standard Linux OS's do not schedule in a deterministic manner because they have non-preemptible kernel sections that introduce an unpredictable amount of scheduler jitter. As such, this project aims to demonstrate that the combination of a PREEMPT_RT Linux kernel with a 5G eMBB (Enhanced Mobile Broadband) Network Slice and a Multi-access Edge Computing (MEC) architecture will allow for a predictable amount of latency between the source and destination .

CHAPTER 2

LITERATURE REVIEW

2.1 Performance Assessment of Linux Kernels with PREEMPT_RT on ARM-Based Embedded Devices

Author and year : George K. Adam, Nikos Petrellis, Lambros T. Doulos, Electronics (MDPI), 2021

Inference : The PREEMPT_RT patch performs well for ARM-based embedded devices, such as single-board computers, as evidenced by using the cyclictst program to determine how quickly the system responds to high-priority tasks. The PREEMPT_RT patch showed an improvement of the worst-case delay and therefore, provides better predictability in the way systems respond. There is no mention of how systems will behave in relation to 5G networks.

2.2 Buildroot Manual – Embedded Linux System Build Tool

Author and year : Buildroot Project Developers, Buildroot Documentation, 2023

Inference : The buildroot project enables developers to create small, efficient Linux systems from scratch. You can use buildroot to create a Linux system by selecting only the specific packages you need (the kernel, libraries, and utilities) and building them all together into a single, compact image. Buildroot is an ideal solution for embedded devices with restricted storage and memory capacity. This document provides an introduction to Buildroot and discusses how to configure the build system, how to choose which packages will be included in the build and how to generate a bootable image for devices such as the Raspberry Pi.

2.3 Linux Kernel Documentation for Raspberry Pi

Author and year : Raspberry Pi Foundation, Raspberry Pi Documentation, 2024

Inference : The raspberry pi foundation provides documentation that describes working with the linux kernel when using raspberry pi hardware. This reference helped us greatly in developing a cross-compiled linux kernel by guiding us through the commands needed to download, configure, and compile the raspberry pi4 64-bit architecture linux kernel using the configurations within menuconfig.

2.4 Performance Evaluation of 5G Standalone Networks for Ultra-Low Latency

Applications

Author and year : C. Guimaraes, A. Ksentini, M. Chen , IEEE Access, 2022

Inference : This paper shows that 5G Standalone networks can achieve very low delay (below 30 ms), making them suitable for real-time applications, but it only focuses on network performance. In our project, we improve this by also considering OS performance using a PREEMPT_RT kernel on Raspberry Pi to measure total system latency.

2.5 A Survey on Mobile Edge Computing: The Communication Perspective

Author and year : Y. Mao, C. You, J. Zhang, K. Huang, K. B. Letaief, IEEE Communications Surveys & Tutorials, 2021

Inference : The second paper explains that Multi-access Edge Computing (MEC) reduces delay by processing data closer to the user instead of sending it to the cloud. This idea is used in our project, where data is sent to a nearby MEC server and displayed in real time, reducing latency.

2.6 Research Gap Identification

While PREEMPT_RT performance on embedded Linux platforms is well documented and the characteristics of latency in 5G networks have been extensively researched; however, no published research exists that evaluates the impact of using a PREEMPT_RT kernel on a Raspberry Pi sending real-time data across an active 5G Standalone network to a Multi-access Edge Computing (MEC) system. This project directly addresses that gap.

CHAPTER 3

OBJECTIVES

3.1 Design of Real-Time Linux System

The objective is to create and implement a Real Time Linux system on a Raspberry Pi 4 utilizing two methods; Building a Lightweight OS based on Linux 6.12.61 + PREEMPT with Buildroot and manually cross-compiling the PREEMPT_RT for Linux Kernel. The goal is to minimize all overhead, ensure a stable booting process, and show a verifiable reduction in worst-case scheduling latency as demonstrated using cyclicttest.

3.2 Integration with 5G Network

To assist in building a reliable 5G Standalone Network using the SIM8200EA-M2 5G HAT device, which will involve ensuring HAT detection, configuring the initial registration to the network, confirming the assignment of an IP address, and testing the performance metrics of the network such as round-trip-performance latency and packet loss.

3.3 Implementation of MEC-Based Log Transfer

To build a complete Real Time logging pipeline that consists of a sender-side python script running on a Raspberry Pi which collects live logs for the Linux System using journalctl, and transfer those logs via UDP ,a Flask application using Socket.IO acting as a MEC and Web Client interface to provide logs to the end-users.

3.4 Performance Evaluation

Performance will be measured for both Kernel builds in terms of scheduling Latency and Jitter, Network Latency using the ping command, CPU utilization, comparing OS image size and comparing the amount of RAM, and end-to-end logging Latency of systems for Real time Industrial/Vehicular/Healthcare Applications.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Overall System Design

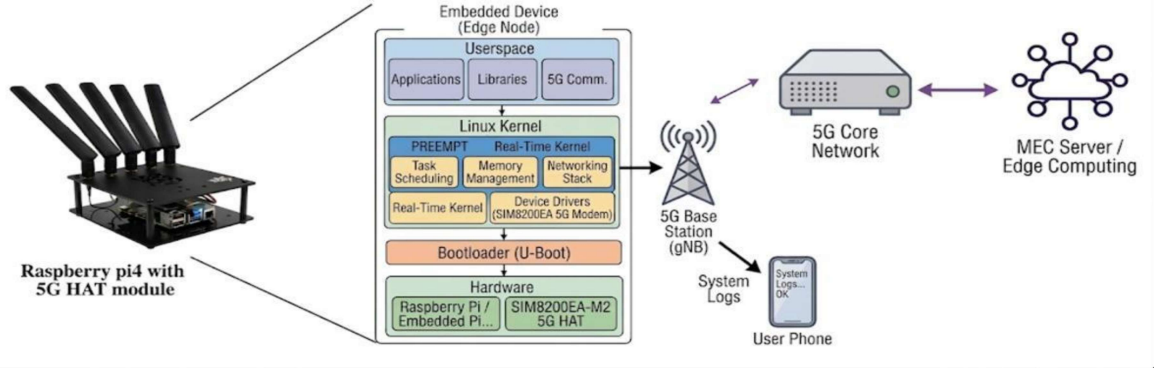


Fig. 4(a): Overall System Architecture

The system's three-functional tiers organize the architecture. The Raspberry Pi 4 with the manually compiled PREEMPT_RT kernel makes up the edge device layer, running the log sender application and sending data to the log receiver application via the 5G HAT. The eMBB slice of the 5G Standalone Network transport layer allows wireless connectivity between the Raspberry Pi and the Multi-access Edge Computing server. The Multi-access Edge Computing server hosts a Flask/Socket.IO application that receives incoming UDP log datagrams and sends them in real-time to the browser clients using WebSockets.

Because there is no overhead per connection and because of the delay imposed by TCP-based connections, UDP was selected rather than TCP.

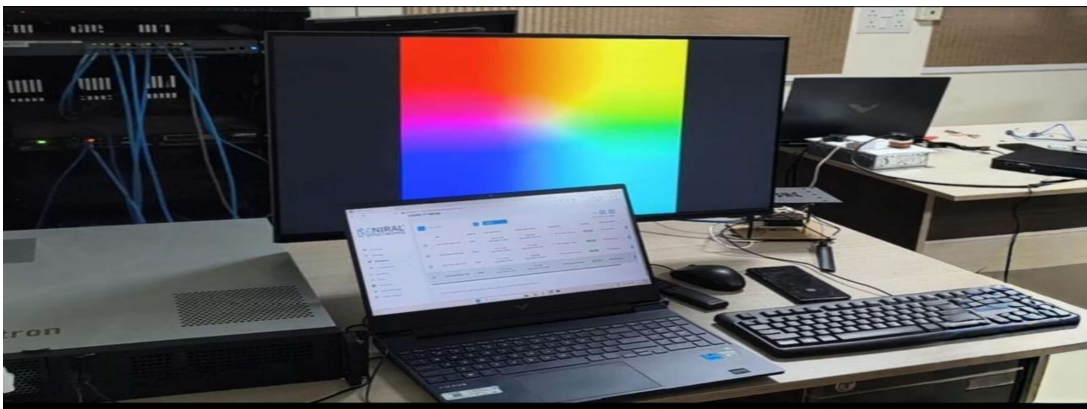


Fig. 4(b): Overall System Setup

4.2 Hardware Components

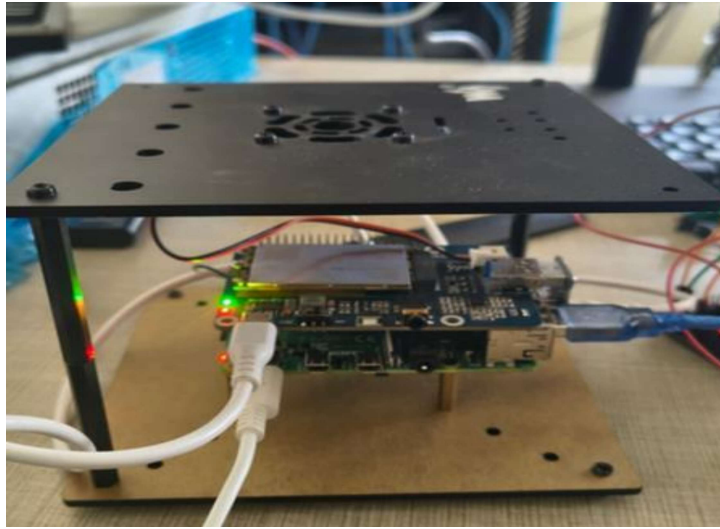


Fig. 4(c): Raspberry Pi 4 with SIM8200EA-M2 5G HAT Hardware Setup

The Raspberry Pi 4 Model B's hardware base includes the Broadcom BCM2711 SoC. This chip consists of an ARM Cortex A72 quad-core processor clocked at 1.8 GHz along with: 4 GB of LPDDR4 RAM; an on-board Gigabit Ethernet Port; 2 on-board USB 3.0 ports; a 40 pin GPIO Header; and other features, all described on the Raspberry Pi website. The BCM2711 has been selected as the hardware platform for cross-compiling the PREEMPT_RT kernel for use with the Raspberry Pi.

The SIM8200EA-M2 5G HAT provides 5G NR Sub 6 GHz connectivity by providing 5G NR bands (n78) along with a 4G LTE fallback connection. The HAT uses USB to connect to the Raspberry Pi. When connected, it will supply the Raspberry Pi with USB network interface and serial interfaces at the Raspberry Pi. During configuration, the HAT supports the use of standard AT command sets for monitoring and configuration purposes.

4.3 Software Architecture

The Software architecture consists of 4 layers. The 1st layer is the hardware abstraction layer. This comprises both the BCM2711 Device Tree Blobs (DTBs) and the device drivers for the BCM2711, which are all loaded at the time of boot. The 2nd Layer is the Kernel layer. This includes either the Buildroot Linux 6.12.61 Kernel or the manually compiled PREEMPT_RT Kernel. The 3rd Layer is the System Service Layer. This includes the init system, networking